

C++

INHERITANCE

A Complete Guide to Object-Oriented Reusability & Hierarchy

Topics: Basics • Types • Access Specifiers • Constructors • Overriding • Polymorphism • Virtual Functions • Abstract Classes

TABLE OF CONTENTS

01

What is Inheritance?

Real-world analogies & motivation

02

Types of Inheritance

Single, Multiple, Multilevel, Hierarchical, Hybrid

03

Access Specifiers

public, private, protected inheritance

04

Base & Derived Classes

Syntax, constructors, destructors

05

Function Overriding

Redefining base class methods

06

Virtual Functions

Runtime polymorphism

07

Abstract Classes

Pure virtual functions & interfaces

08

Real-Life Examples

Employee, Vehicle, Shape hierarchies

09

Best Practices

Design patterns & pitfalls

SECTION 03

Access Specifiers

public • private • protected — Controlling Visibility in Inheritance

INHERITANCE TYPES — BASED ON ACCESS SPECIFIER

- Based on Access Specifier the inheritance has three types

private

protected

public

1. Public Inheritance

- Public inheritance is specified as →
- Base class private members
 - Not accessible directly
 - Still inherited
 - Manipulate through inherited member functions
- Base class public and protected members
 - Inherited with original member access
- friend functions
 - Not inherited

```
class derived_class_name : public base_class_name
{
    // data members and member functions of class A
}
```

INHERITANCE TYPES — BASED ON ACCESS SPECIFIER

private

protected

public

2. Protected Inheritances

- Protected inheritance is specified as →
- Intermediate level of protection between public and private
- Protected members accessible to
 - Base class members
 - Base class friends
 - Derived class members
 - Derived class friends
- Derived-class members
 - Refer to public and protected members of base class
 - Simply use member names

```
class derived_class_name : protected base_class_name  
{  
    // data members and member functions of class A  
}
```

Protected members of a class can be accessed within the class itself and within its derived classes, but **not through objects**. Access to protected members must happen inside member functions of the base or derived class.

INHERITANCE TYPES — BASED ON ACCESS SPECIFIER

private

protected

public

3. Private Inheritance

- Private inheritance is specified as →
- Public and protected become **private members of the derived class**
- If we do not use specifier, then by default the private inheritance is used

```
class derived_class_name : private base_class_name  
{  
    // data members and member functions of class A  
}
```

A base class's private members are never accessible directly from a derived class, but can be accessed through calls to the public and protected members of the base class

ACCESS SPECIFIERS IN INHERITANCE

How base class member access changes in derived class:

Base Class Member	public Inheritance	protected Inheritance	private Inheritance
public member	public	protected	private
protected member	protected	protected	private

Key Rules to Remember:

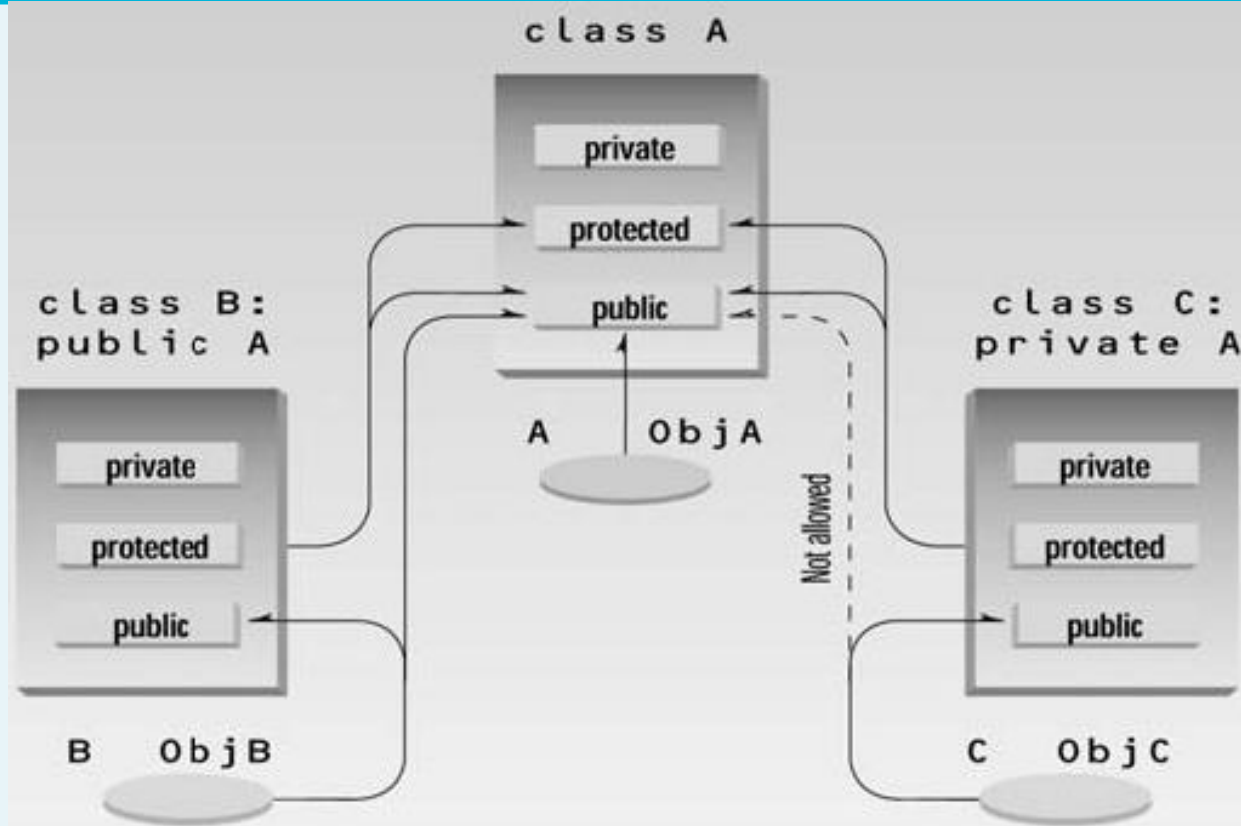
- public: Best for 'is-a' relationships. Maintains original access levels. Most commonly used.
- protected: Inherited members become protected — accessible in derived class but not outside.
- private: Everything inherited becomes private — next level derived classes can't access even protected members.
- private members are NEVER directly accessible in derived class regardless of inheritance type.

Public and Private Inheritance

class manager : public employee

- keyword **public** specifies that objects of the derived class are able to access public member functions of the base class.
- If **private** is used in place of public that means objects of the derived class cannot access public member functions of the base class, they can only access public member of their own derived class.
- Since **objects can never access private or protected members of a class**, the result is that **no member of the base class is accessible to objects of the derived class**.
- If you don't supply any access specifier when creating a class, private is assumed.

Public and Private Inheritance



Public and Private Inheritance – Example [1/2]

```
// tests publicly- and privately-derived classes
#include <iostream>
using namespace std;
class A{                                // base class
private:    int privdataA;              // (functions have the same access
protected: int protdataA;             // rules as the data shown here)
public:     int pubdataA;
};
class B : public A{                     // publicly-derived class
public:
void funct(){
    int a;
    a = privdataA;                      // error: not accessible
    a = protdataA;                      // OK
    a = pubdataA;                      // OK
}
};
```

Public and Private Inheritance – Example [2/2]

```
class C : private A{           // privately-derived class
public:
    void funct(){
        int a;
        a = privdataA;         // error: not accessible
        a = protdataA;         /* OK */
        a = pubdataA;          // OK
    } };
int main(){
    int a; B objB; a = objB.privdataA; // error: not accessible
    a = objB.protdataA;               // error: not accessible
    a = objB.pubdataA;                // OK (A public to B)
    C objC; a = objC.privdataA;       // error: not accessible
    a = objC.protdataA;               // error: not accessible
    a = objC.pubdataA; // error: not accessible (A private to C)
    return 0;
}
```

COMPLETE EXAMPLE: All Access Types Compared

```
class Base {
private:
    int privateVar = 1;    // Base only
protected:
    int protectedVar = 2;  // Base + Derived
public:
    int publicVar = 3;     // Anyone

    void show() {
        cout << privateVar;    // ✔
        cout << protectedVar;  // ✔
        cout << publicVar;     // ✔
    }
};

class PublicDerived : public Base {
    void demo() {
        // privateVar = 10;    // ✗ Cannot access
        protectedVar = 10;    // ✔ Protected → stays protected
        publicVar = 20;       // ✔ Public → stays public
    }
};

class ProtectedDerived : protected Base {
    void demo() {
        // privateVar = 10;    // ✗ Cannot access
        protectedVar = 10;    // ✔ Protected → stays protected
        publicVar = 20;       // ✔ Public → becomes protected
    }
};
```

```
class PrivateDerived : private Base {
    void demo() {
        // privateVar = 10;    // ✗ Cannot access
        protectedVar = 10;    // ✔ Protected → becomes private
        publicVar = 20;       // ✔ Public → becomes private
    }
};

int main() {
    PublicDerived pd;
    // pd.protectedVar = 5;    // ✗ protected = not outside
    pd.publicVar = 5;         // ✔ public in PublicDerived
    // pd.publicVar = 5; // ProtectedDerived/PrivateDerived would be ✗
}
```

Base class member access specifier	Type of inheritance		
	Public inheritance	Protected inheritance	private inheritance
Public	public in derived class. Can be accessed directly by any non-static member functions, friend functions and non-member functions.	protected in derived class. Can be accessed directly by all non-static member functions and friend functions.	private in derived class. Can be accessed directly by all non-static member functions and friend functions.
Protected	protected in derived class. Can be accessed directly by all non-static member functions and friend functions.	protected in derived class. Can be accessed directly by all non-static member functions and friend functions.	private in derived class. Can be accessed directly by all non-static member functions and friend functions.
Private	Hidden in derived class. Can be accessed by non-static member functions and friend functions through public or protected member functions of the base class.	Hidden in derived class. Can be accessed by non-static member functions and friend functions through public or protected member functions of the base class.	Hidden in derived class. Can be accessed by non-static member functions and friend functions through public or protected member functions of the base class.

Accessing Base Class Members

Access Specifier	Accessible from own class	Accessible from derived class	Accessible from objects outside class
Public:	Yes	Yes	Yes
Protected:	Yes	Yes	No
Private;	Yes	No	No

UNDERSTANDING protected MEMBERS

protected = accessible within class AND its derived classes, but NOT outside

private

Accessible ONLY
within the class itself

protected

Accessible within class
AND all derived classes

public

Accessible from
ANYWHERE

```
class Base {
private:   int x;           // Only Base can access x
protected: int y;         // Base + Derived can access y
public:   int z;           // Anyone can access z
};

class Derived : public Base {
    void demo() {
        // x = 5;    // ✗ ERROR: private, not accessible
        y = 10;      // ✔ OK: protected accessible in derived
        z = 20;      // ✔ OK: public accessible everywhere
    }
};

int main() {
    Derived d;
    // d.y = 5;      // ✗ ERROR: protected not accessible outside
    d.z = 20;        // ✔ OK: public accessible outside
}
```

SECTION 04

Constructors & Destructors

Initialization order, base-class calls, RAI in derived classes

Constructors and Destructors in Derived Classes

- Instantiating a derived-class object begins a chain of constructors
- Derived class constructor invokes base class constructor either explicitly (via a base-class member initializer) or implicitly (calling the base class's default constructor)
- Constructor of the derived class called first and finishes last
- Constructor of the base class of the hierarchy is called last in the chain but is the first constructor body to finish
- The original derived-class constructor's body finishes executing last
- This process cascades up the hierarchy if it contains more than two levels
- Both base and derived class constructors execute automatically

Constructors in Derived Classes

- If the constructors have arguments, the syntax of the derived constructor class is different, as below

```
class derived_class_name : public base_class_name
{
    derived_class_constructor(args):base_class_constructor(args)
}
```

- In case of multiple inheritance the syntax will be as

```
class derived_class_name : public base_class1,public base_class2
{
    derived_class_constructor(args): base_class1_constructor(args) ,
                                     base_class2_constructor(args)
}
```

- Calling constructor of the base class from child class constructor in this manner is called base class initialization

Constructors in Derived Classes

```
class Parent{
public:
    Parent(){
        cout<<"Parent Constructor...";
    }
    ...
};

class Child : public Parent{
public:
    Child():Parent()
    {
        cout<<"Child Constructor...";
    }
    ...
};
```

Constructors in Derived Classes

- User can also provide base class initializer and member initializer simultaneously

```
class Parent{
    public:
        Parent() {...}
};
class Child : public Parent{
    int member;
    public:
        Child():member(0) , Parent() //data member of the derived
                                     //class and constructor of
                                     //the base class initialized
                                     //simultaneously
        {...}
};
```

- Derived class can not initialize the data members of the base class using member initialization list, it must initialize these members using base class constructor

Derived Class Constructors – [1/2]

```
// constructors in derived class
#include <iostream>
using namespace std;
class Counter{
protected:                                // NOTE: not private
    unsigned int count;                    // count
public:
    Counter() : count(0) {}                // constructor, no args
    Counter(int c) : count(c) {}           // constructor, one arg
    unsigned int get_count() const         // return count
    { return count; }
    Counter operator ++ ()                 // incr count (prefix)
    { return Counter(++count); }
};
class CountDn : public Counter{
public: CountDn() : Counter() {}           // constructor, no args
```

Derived Class Constructors – [1/2]

```
CountDn(int c) : Counter(c) {}           // constructor, 1 arg
CountDn operator -- ()                   // decr count (prefix)
{ return CountDn(--count); }
};
int main(){
    CountDn c1; CountDn c2(100);
    cout << "\nc1=" << c1.get_count();    // display
    cout << "\nc2=" << c2.get_count();    // display
    ++c1; ++c1; ++c1;                     // increment c1
    cout << "\nc1=" << c1.get_count();    // display it
    --c2; --c2;                           // decrement c2
    cout << "\nc2=" << c2.get_count();    // display it
    CountDn c3 = --c2;                     // = calls 1 arg constructor
    cout << "\nc3=" << c3.get_count();    // display c3
    cout << endl;
    return 0;
}
```

Destructors in Derived Classes

- Destructors are called in reverse order of the constructors
- When a derived-class object is destroyed, the program calls that object's destructor
- Destructors of the derived class called first
- Destructors of the next base class up hierarchy in called next
- This process repeats until the destructor of the final base class at the top of the hierarchy is called
- Then the object is removed from memory
- Base-class constructors, destructors and overloaded assignment operators are not inherited by derived classes
- Derived-class constructors, destructors and overloaded assignment operators, however, can call base-class versions

CONSTRUCTOR CHAINING IN INHERITANCE

1 Derived Constructor Called

Stack frame created for Child

2 Base Constructor Invoked

Child calls Parent(args) via initializer list

3 Base Body Executes First

Parent's constructor body runs completely

4 Derived Body Executes

Child's constructor body runs next

CONSTRUCTOR

```
class Animal {
    string name;
public:
    Animal(string n) : name(n) {
        cout << "Animal built: " << name;
    }
    ~Animal() {
        cout << "Animal destroyed";
    }
};

class Dog : public Animal {
    string breed;
public:
    Dog(string n, string b)
        : Animal(n), breed(b) {
        cout << "Dog built: " << breed;
    }
    ~Dog() {
        cout << "Dog destroyed";
    }
};
```

OUTPUT

```
// Dog d("Rex", "Labrador");
// CONSTRUCTION ORDER:
Animal built: Rex
Dog built: Labrador

// Object goes out of scope:
// DESTRUCTION ORDER (REVERSE):
Dog destroyed
Animal destroyed
```


Constructors in Multiple Inheritance

(1/5)

```
// multiple inheritance with English Distances
#include <iostream>
#include <string>
using namespace std;
class Type{                                // type of lumber
private:
    string dimensions; string grade;
public:
    Type() : dimensions("N/A"), grade("N/A")
    { }                                     // no-arg constructor
    Type(string di, string gr) : dimensions(di), grade(gr)
    { }                                     // 2-arg constructor
    void gettype(){                         // get type from user
        cout << "    Enter nominal dimensions (2x4 etc.): ";
        cin >> dimensions;
        cout << "    Enter grade (rough, const, etc.): ";
        cin >> grade; }
}
```

Constructors in Multiple Inheritance

(2/5)

```
void showtype() const{           // display type
    cout << "\n  Dimensions: " << dimensions;
    cout << "\n  Grade: " << grade;
}
};

class Distance{                 // English Distance class
private: int feet;    float inches;
public:                        // no-arg constructor
    Distance() : feet(0), inches(0.0)
    { }                       // constructor (two args)
    Distance(int ft, float in) : feet(ft), inches(in)
    { }
    void getdist(){           // get length from user
        cout << "    Enter feet: ";    cin >> feet;
        cout << "    Enter inches: ";  cin >> inches;
    }
```

Constructors in Multiple Inheritance

(3/5)

```
void showdist() const           // display distance
{ cout << feet << "\'-" << inches << '\\"'; }
};

class Lumber : public Type, public Distance {
private:
    int quantity;               // number of pieces
    double price;               // price of each piece
public:                         // constructor (no args)
    Lumber() : Type(), Distance(), quantity(0), price(0.0)
    { }                         // constructor (6 args)
    Lumber( string di, string gr, // args for Type
            int ft, float in,    // args for Distance
            int qu, float prc ) : // args for our data
        Type(di, gr),           // call Type ctor
        Distance(ft, in),       // call Distance ctor
        quantity(qu), price(prc) // initialize our data
    { }
```

Constructors in Multiple Inheritance

(4/5)

```
void getlumber()
{
    Type::gettype();
    Distance::getdist();
    cout << "    Enter quantity: ";      cin >> quantity;
    cout << "    Enter price per piece: "; cin >> price;
}

void showlumber() const
{
    Type::showtype();
    cout << "\n    Length: ";
    Distance::showdist();
    cout << "\n    Price for " << quantity
         << " pieces: $" << price * quantity;
}

};
```

Constructors in Multiple Inheritance

(5/5)

```
int main()
{
    Lumber siding;                // constructor (no args)

    cout << "\nSiding data:\n";
    siding.getlumber();           // get siding from user

                                // constructor (6 args)
    Lumber studs( "2x4", "const", 8, 0.0, 200, 4.45F );

                                // display lumber data
    cout << "\nSiding";  siding.showlumber();
    cout << "\nStuds";   studs.showlumber();
    cout << endl;
    return 0;
}
```

What Is Inherited?

MOST BUT NOT ALL

INHERITED

- ✓ Data members
- ✓ Most member functions

NOT INHERITED

- ✓ Constructors
- ✓ Destructors
- ✓ Friend functions

SECTION 05

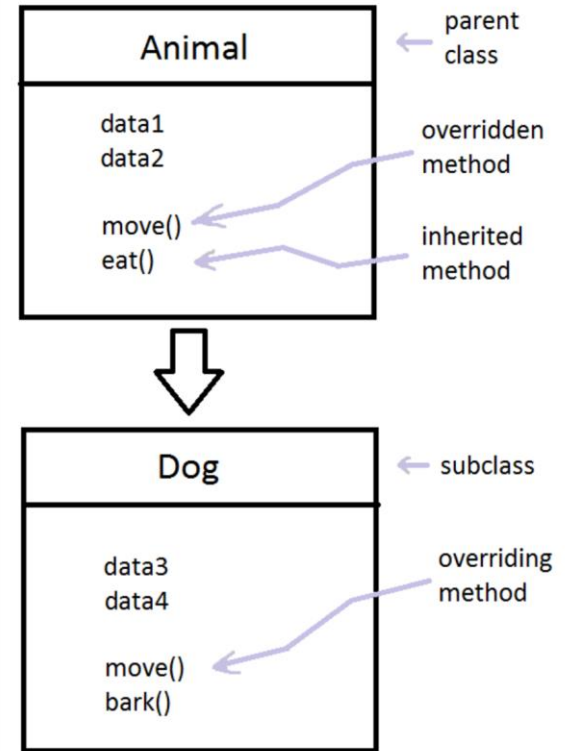
Function Overriding

Redefining base class methods in derived class with same signature

Overriding Member Functions of Base Class

- Derived class can override the member functions of its base class
- To override a function the derived class simply provides a function with the **same signature and return type** as that of its base class

In simple words, redefining a function of the base class in a derived class with the same signature and return type but with a different definition is called function overriding



Overriding Member Functions of Base Class

- When the same function exists in both the base class and the derived class, the function in the derived class will be executed.

```
DateTime dt(12,3,2001,12,30);  
dt.Display(); // Display of derived class DateTime is called
```

- One can use the scope resolution operator to specify exactly what class the function is a member of

```
void DateTime::Display()  
{  
    Date::Display(); // explicitly calls Display() of class Date  
    cout << " " << hours << ":" << minutes;  
}
```

Overriding Member Functions

(1/3)

```
// overloading functions in base and derived classes
#include <iostream>
using namespace std;
#include <process.h>           // for exit()
class Stack
{
protected:                   // NOTE: can't be private
    enum { MAX = 3 };         // size of stack array
    int st[MAX];              // stack: array of integers
    int top;                  // index to top of stack
public:
    Stack() { top = -1; }     // constructor
    void push(int var)        // put number on stack
    { st[++top] = var; }
    int pop()                 // take number off stack
    { return st[top--]; }
};
```

Overriding Member Functions

(2/3)

```
class Stack2 : public Stack{
public:
    void push(int var)           // put number on stack
    {
        if(top >= MAX-1)         // error if stack full
        { cout << "\nError: stack is full";
          exit(1);
        }
        Stack::push(var);        // call push() in Stack class
    }
    int pop()                    // take number off stack
    {
        if(top < 0)              // error if stack empty
        { cout << "\nError: stack is empty\n";
          exit(1); }
        return Stack::pop();     // call pop() in Stack class
    }
}
```

Overriding Member Functions

(3/3)

```
    }  
};  
  
int main()  
{  
    Stack2 s1;  
    s1.push(11);           // push some values onto stack  
    s1.push(22);  
    s1.push(33);  
    cout << endl << s1.pop(); // pop some values from stack  
    cout << endl << s1.pop();  
    cout << endl << s1.pop();  
    cout << endl << s1.pop(); // oops, popped one too many...  
    cout << endl;  
    return 0;  
}
```

OVERRIDING vs OVERLOADING

Function OVERLOADING

- ✓ Same class, different parameters
- ✓ Compile-time (static) polymorphism
- ✓ Same function name, different signature
- ✓ Return type can differ
- ✓ Does NOT require inheritance

```
class Calc {  
public:  
    int add(int a, int b)  
    { return a+b; }  
    // Same class, diff params:  
    float add(float a, float b)  
    { return a+b; }  
    int add(int a, int b, int c)  
    { return a+b+c; }  
};
```

Function OVERRIDING

- ✓ Derived class redefines base method
- ✓ Runtime polymorphism (with virtual)
- ✓ SAME name AND same signature
- ✓ Return type must be same
- ✓ REQUIRES inheritance

```
class Shape {  
public:  
    void draw() {  
        cout << "Drawing shape";  
    }  
};  
class Circle : public Shape {  
public:  
    void draw() { // OVERRIDING  
        cout << "Drawing circle";  
    }  
};
```

REAL-WORLD OVERRIDING: Shape System

SHAPE OVERRIDING EXAMPLE

```
class Shape {
public:
    void draw() {
        cout << "Drawing shape\n";
    }
    float area() {
        return 0.0f;
    }
};

class Circle : public Shape {
private:
    float radius;
public:
    void draw() {          // OVERRIDES
        cout << "Drawing circle\n";
    }
    float area() {         // OVERRIDES
        return 3.14f * radius * radius;
    }
};

class Rectangle : public Shape {
private:
    float width, height;
public:
    void draw() {          // OVERRIDES
        cout << "Drawing rectangle\n";
    }
    float area() {         // OVERRIDES
        return width * height;
    }
};

int main() {
    Shape s;    s.draw();    // "Drawing shape"
    Circle c;   c.draw();    // "Drawing circle"
    Rectangle r; r.draw();   // "Drawing rectangle"
    // NOTE: Without 'virtual', calling via
    // base pointer calls BASE version!
    Shape* ptr = new Circle();
    ptr->draw(); // "Drawing shape"!! NOT circle
}              // (need virtual for polymorphism)
```

SECTION 06

Abstract Classes

Interfaces • = 0 • Cannot be instantiated

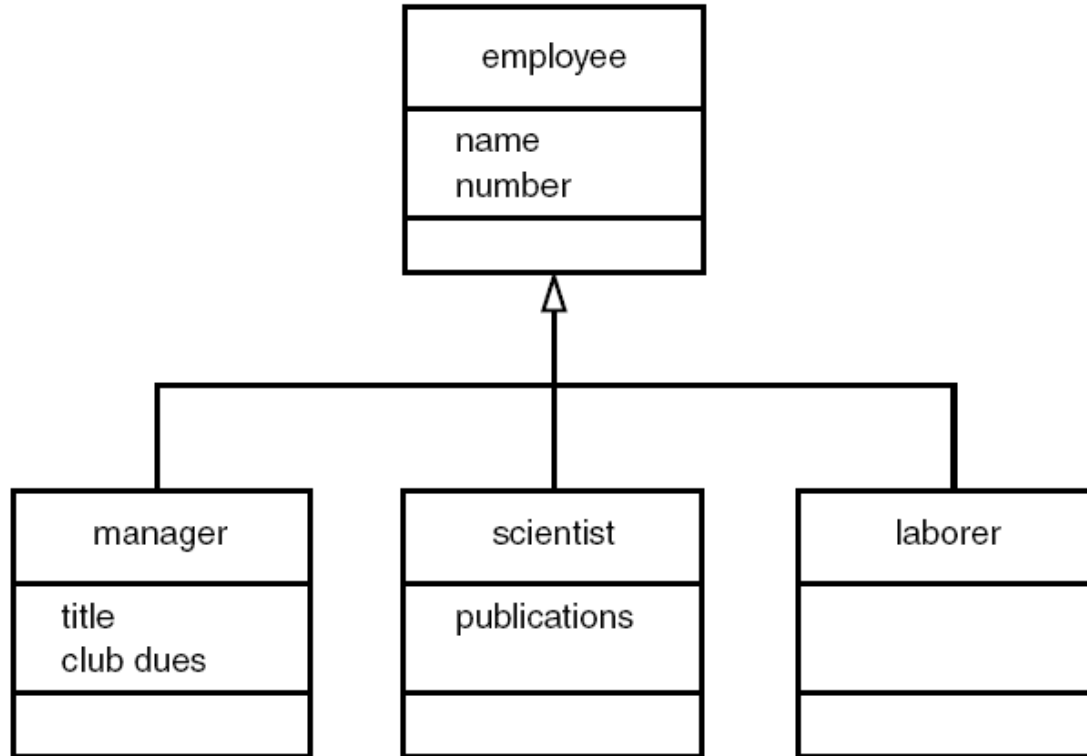
Abstract Classes

- Suppose you are assigned to create a **database of employees** of a widget company.
- For simplicity, only three kinds of employees are represented.
 - **Managers** manage
 - **Scientists** perform research to develop better widgets
 - **Laborers** operate the dangerous widget-stamping presses
- The database stores a name and an employee identification number for all employees, no matter what their category.
 - For managers, it also stores their titles and golf club dues.
 - For scientists, it stores the number of scholarly articles they have published.
 - Laborers need **no additional data** beyond their names and numbers.

Abstract Classes

- Our example program starts with a base class employee.
 - This class handles the employee's last name and employee number.
- From this class three other classes are derived:
 - Manager
 - Scientist
 - Laborer
- The manager and scientist classes contain additional information about these categories of employee, and member functions to handle this information,

Abstract Classes



'Abstract' Base Class

- **Employee** is a general class whose sole purpose is to act as a base from which other classes are derived.
 - don't define any objects of the base class employee
- The **laborer** class operates identically to the employee class, since it contains no additional data or functions.
- **It may seem that the laborer class is unnecessary**, but by making it a separate class
 - we emphasize that all classes are descended from the same source, employee.
 - if in the future we modify the laborer class, we would not need to change the employee class.
- Classes used only for deriving other classes, as employee, are sometimes loosely called **abstract classes**, meaning that **no actual instances (objects) of this class are created**.

Class Hierarchies (Employee Database)

(1/4)

```
// models employee database using inheritance
#include <iostream>
using namespace std;
const int LEN = 80;           // maximum length of names
class employee {              // employee class
private:
    char name[LEN];           // employee name
    unsigned long number;     // employee number
public:
    void getdata(){
        cout << "\n Enter last name: ";    cin.get(name,LEN);
        cout << " Enter number: ";    cin >> number;
    }
    void putdata() const{
        cout << "\n Name: " << name;
        cout << "\n Number: " << number;
    }
};
```

Class Hierarchies (Employee Database)

(2/4)

```
class manager : private employee{           // management class
private:
    char title[LEN];                        // "vice-president" etc.
    double dues;                            // golf club dues
public:
    void getdata(){
        employee::getdata();
        cout << " Enter title: "; cin.get(title,LEN);
        cout << " Enter golf club dues: "; cin >> dues;
    }
    void putdata() const{
        employee::putdata();
        cout << "\n Title: " << title;
        cout << "\n Golf club dues: " << dues;
    }
};
```

Class Hierarchies (Employee Database)

(3/4)

```
class scientist : private employee{    // scientist class
private:
    int pubs;                        // number of publications
public:
    void getdata()
    {
        employee::getdata();
        cout << " Enter number of pubs: "; cin >> pubs;
    }
    void putdata() const
    {
        employee::putdata();
        cout << "\n Number of publications: " << pubs;
    }
};
class laborer : public employee        // laborer class
{   };
```

Class Hierarchies (Employee Database)

(4/4)

```
int main(){
    manager m1, m2;      scientist s1;      laborer l1;

    cout << endl;          // get data for several employees
    cout << "\nEnter data for manager 1";      m1.getdata();
    cout << "\nEnter data for manager 2";      m2.getdata();
    cout << "\nEnter data for scientist 1";      s1.getdata();
    cout << "\nEnter data for laborer 1";      l1.getdata();

    //display data for several employees
    cout << "\nData on manager 1";      m1.putdata();
    cout << "\nData on manager 2";      m2.putdata();
    cout << "\nData on scientist 1";      s1.putdata();
    cout << "\nData on laborer 1";      l1.putdata();
    cout << endl;
    return 0;
}
```

SECTION 07

Virtual Functions & Pure Virtual Functions

Runtime dispatch • vtable • dynamic binding • override keyword

Abstract Classes

- A **virtual function** is a member function **declared in a base class** that can be **overridden in derived classes**.
- When called through a **base class pointer or reference**, the program decides *at runtime* which version to call **based on the actual object type, not the pointer type**.
 - This is called **dynamic dispatch** (or runtime polymorphism).

VIRTUAL FUNCTIONS

virtual keyword enables **runtime polymorphism**. When a base class pointer/reference points to a derived object, calling a virtual function dispatches to the derived class version at **runtime** (not compile time).

```
class Animal {
public:
    virtual void speak() {
        // VIRTUAL: enables runtime dispatch
        cout << "Animal sound\n";
    }
};

class Dog : public Animal {
public:
    void speak() override {
        // override keyword: compile-time safety check
        cout << "Woof!\n";
    }
};

class Cat : public Animal {
public:
    void speak() override {
        cout << "Meow!\n";
    }
};

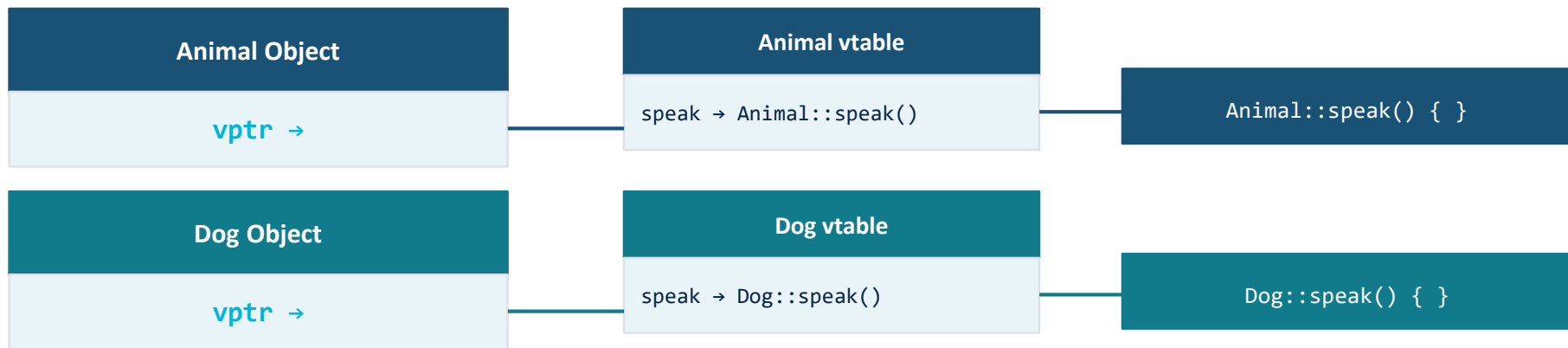
int main() {
    Animal* animals[3];
    animals[0] = new Animal();      // Base object
    animals[1] = new Dog();          // Derived - Dog
    animals[2] = new Cat();          // Derived - Cat

    for (int i = 0; i < 3; i++) {
        animals[i]->speak(); // Calls correct version at RUNTIME!
    }
}

//same call syntax, different behavior - "polymorphic call"
```

Output: Animal sound
Woof!
Meow!

HOW VIRTUAL FUNCTIONS WORK: vtable



How it works:

- Every class with virtual functions gets a vtable (virtual function table) — an array of function pointers.
- Each object of that class stores a hidden vptr (virtual pointer) pointing to its class's vtable.
- When you call a virtual function through a pointer/reference, the runtime follows `vptr → vtable → correct function`.
- This is why virtual calls have a tiny overhead vs. non-virtual calls (one extra indirection).

Pure Virtual Function

- A **pure virtual function** has no body and *must* be overridden in derived classes. Any class containing one becomes **abstract** (cannot be instantiated).
- Derived classes **must implement** this function

```
class Shape
```

```
{
```

```
public:
```

```
virtual double area() = 0; // pure virtual — "= 0" syntax
```

```
};
```

```
class Circle : public Shape
```

```
{
```

```
double r;
```

```
public:
```

```
Circle(double r) : r(r)
```

```
{}
```

```
double area() override {
```

```
return 3.14159 * r * r;
```

```
}
```

```
};
```

```
// Shape s; // ✗ Error: can't instantiate abstract class
```

```
Shape* s = new Circle(5); // ✓ Fine
```

Virtual Destructor

- When you delete a derived class object through a base class pointer, the compiler needs to know which destructor to call.
- Without virtual, it always calls the base class destructor only — leaving the derived part partially destroyed.
- Always declare the destructor virtual in a polymorphic base class.
 - Without virtual, it always calls the base class destructor only — leaving the derived part partially destroyed (**memory leak**).

```
class Base {  
public:  
    virtual ~Base() { cout << "Base destroyed\n"; }  
};  
  
class Derived : public Base {  
public:  
    ~Derived() { cout << "Derived destroyed\n"; }  
};  
  
Base* b = new Derived();  
delete b;    // Correctly calls ~Derived(), then  
             ~Base()
```

ABSTRACT CLASSES & PURE VIRTUAL FUNCTIONS

A class with at least one **pure virtual function** (= 0) is called an **abstract class**. It cannot be instantiated directly — it serves as a blueprint that derived classes must complete.

```
// Abstract base class – cannot be instantiated
class Shape {                // "Shape" object alone is meaningless
public:
    virtual float area() = 0;    // Pure virtual: MUST be overridden
    virtual float perimeter() = 0; // Pure virtual: MUST be overridden
    virtual void draw() = 0;    // Pure virtual: MUST be overridden

    void display() {           // Regular (non-pure) virtual OK
        cout << "Area: " << area() << ", Perimeter: " << perimeter();
    }
};

class Circle : public Shape { // Must implement ALL pure virtuals
public:
    float radius;
    Circle(float r) : radius(r) {}
    float area() override { return 3.14f * radius * radius; }
    float perimeter() override { return 2 * 3.14f * radius; }
    void draw() override { cout << "Drawing Circle with r=" << radius; }
};
```

```
class Rectangle : public Shape {
public:
    float w, h;
    Rectangle(float w, float h) : w(w), h(h) {}
    float area() override { return w * h; }
    float perimeter() override { return 2*(w+h); }
    void draw() override { cout << "Drawing Rectangle " << w <<
"x" << h; }
};

// Shape s;           // ✗ ERROR: Cannot instantiate abstract class
// Shape* p = &s;     // ✗ ERROR
Shape* s1 = new Circle(5); // ✔ Pointer to derived OK
Shape* s2 = new Rectangle(4,6); // ✔
s1->display();           // polymorphic call
s2->display();
```

Rule: If a derived class doesn't implement ALL pure virtuals, it also becomes abstract and cannot be instantiated.

SECTION 08

Real-Life Examples

Employee System • Banking • Vehicle Fleet • GUI Toolkit

REAL EXAMPLE 1: Employee Management System

```
class Employee {                                // BASE CLASS
protected:
    string name, id;
    float salary;
public:
    Employee(string n, string i, float s) : name(n), id(i), salary(s) {}
    virtual float calcBonus() { return salary * 0.05f; } // 5% default bonus
    virtual void display() {
        cout << "Employee: " << name << ", ID: " << id
            << ", Salary: " << salary << ", Bonus: " << calcBonus();
    }
};

class Manager : public Employee {                // IS-A Employee
    int teamSize;
public:
    Manager(string n, string i, float s, int t)
        : Employee(n,i,s), teamSize(t) {}
    float calcBonus() override { // Managers get 15% + team incentive
        return salary * 0.15f + teamSize * 500;
    }
    void display() override {
        Employee::display();           // Call base first
        cout << ", Team Size: " << teamSize;
    }
};
```

```
class Developer : public Employee { // IS-A Employee
    string language;
public:
    Developer(string n, string i, float s, string l)
        : Employee(n,i,s), language(l) {}
    float calcBonus() override {
        return salary * 0.10f; }        // 10% for devs
};

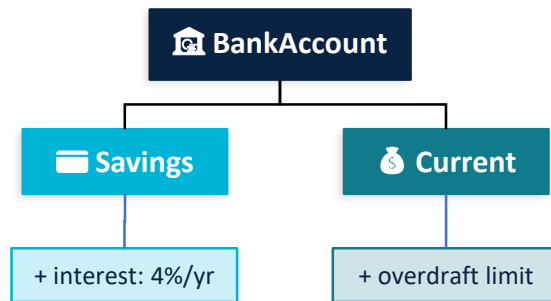
int main() {
    Employee* staff[3];
    staff[0] = new Employee("Ali", "E001", 50000);
    staff[1] = new Manager("Sara", "M001", 80000, 8);
    staff[2] = new Developer("Zain", "D001", 70000, "C++");

    for (auto* e : staff) e->display(); // Polymorphic call!
}
```


REAL EXAMPLE 2: Banking System

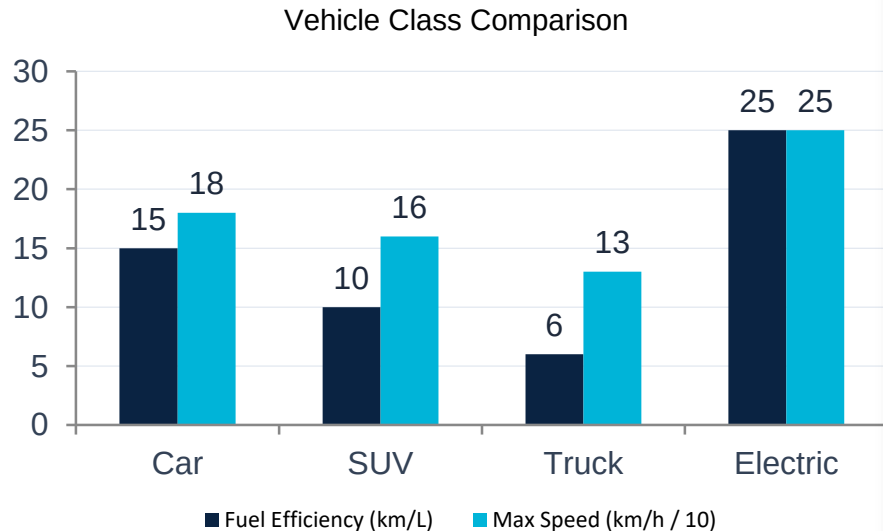
```
class BankAccount {
protected: string owner; float balance;
public:
    BankAccount(string o, float b) : owner(o), balance(b)
    {}
    virtual void deposit(float amt) { balance += amt; }
    virtual bool withdraw(float amt) {
        if (amt > balance) return false;
        balance -= amt; return true;
    }
    virtual void showBalance() { cout << owner << ": Rs."
        << balance; }
};

class SavingsAccount : public BankAccount {
    float interestRate;
public:
    SavingsAccount(string o, float b, float r) :
    BankAccount(o,b), interestRate(r) {}
    void addInterest() {
        balance += balance * interestRate; }
};
```



```
class CurrentAccount : public BankAccount {
    float overdraftLimit;
public:
    CurrentAccount(string o, float b, float od) :
    BankAccount(o,b), overdraftLimit(od) {}
    bool withdraw(float amt) override {
        // Can go below zero up to overdraft limit
        if (amt > balance + overdraftLimit)
            return false;
        balance -= amt;
        return true;
    }
};
```

REAL EXAMPLE 3: Vehicle Fleet Management

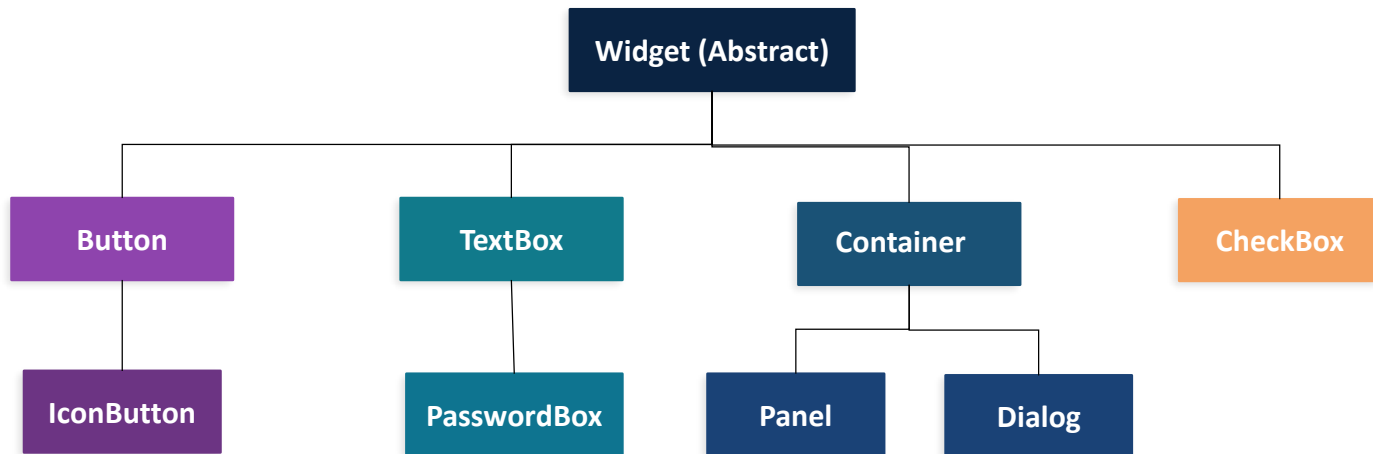


```
class Vehicle {
protected:
    string model; float speed;
    virtual float fuelEfficiency() = 0; // Pure virtual
public:
    Vehicle(string m, float s) : model(m), speed(s) {}
    virtual void start() { cout << model << " starting..."; }
    virtual void brake() { cout << model << " braking..."; }
    void stats() {
        cout << model << " | Max: " << speed
            << "km/h | Fuel: " << fuelEfficiency() << "km/L";
    }
};

class Car : public Vehicle {
    int doors;
public:
    Car(string m, float s, int d) : Vehicle(m,s), doors(d) {}
    float fuelEfficiency() override { return 15.0f; }
};

class ElectricCar : public Car {
    float batteryKm;
public:
    ElectricCar(string m, float s, float bk)
        : Car(m,s,4), batteryKm(bk) {}
    float fuelEfficiency() override { return 25.0f; }
    void start() override {
        cout << model << " (Electric) starting silently";
    }
};
```

REAL EXAMPLE 4: GUI Toolkit (Widget Hierarchy)



Why Inheritance for GUI?

- All widgets share: position, size, color, render(), handleEvent() — defined once in Widget base
- Button, TextBox etc. override render() and handleEvent() for specific behavior
- A renderer loop can iterate over Widget* array and call render() polymorphically

REAL EXAMPLE 5: University Library System

```
// Abstract base class
class LibraryItem {
protected: string title, author; bool isAvailable;
public:
    LibraryItem(string t, string a)
        : title(t), author(a), isAvailable(true) {}
    virtual void display() const = 0;           // Pure virtual
    virtual double lateFee(int days) const = 0; // Pure virtual
    virtual bool checkout() {
        if(!isAvailable) return false;
        isAvailable = false; return true;
    }
    virtual ~LibraryItem() {}
};

class Book : public LibraryItem {
    int pages; string ISBN;
public:
    Book(string t, string a, int p, string i)
        : LibraryItem(t,a), pages(p), ISBN(i) {}
    void display() const override {
        cout << "📖 Book: " << title << " by " << author
            << " | Pages: " << pages;
    }
    double lateFee(int days) const override { return days * 5.0; } // };
Rs.5/day
};

class DVD : public LibraryItem {
```

```
public:
    DVD(string t, string a, int d, string g)
        : LibraryItem(t,a), duration(d), genre(g) {}
    void display() const override {
        cout << "🎬 DVD: " << title << " | Duration: " << duration <<
"min";
    }
    double lateFee(int days) const override { return days * 10.0; } //
Rs.10/day
};

class EBook : public Book {                // EBook IS-A Book!
    string format; float sizeGB;
public:
    EBook(string t, string a, int p, string i, string f, float s)
        : Book(t,a,p,i), format(f), sizeGB(s) {}
    void display() const override {
        Book::display();
        cout << " | Format: " << format << " | Size: " << sizeGB <<
"GB";
    }
    bool checkout() override {              // EBooks always available
        return true;
    }
};
```

SECTION 09

Best Practices & Common Pitfalls

Design principles • Diamond problem • virtual destructors

BEST PRACTICES IN INHERITANCE



Always use virtual destructor in base class

Without it, deleting via base pointer leaks derived class resources

```
virtual ~Base() {}
```



Use 'override' keyword in derived class

Compiler catches signature mismatches; prevents silent bugs

```
void draw() override;
```



Prefer public inheritance for 'is-a'

Matches semantic intent; enables polymorphism correctly

```
class Dog : public Animal {}
```



Call base class functions explicitly when needed

Avoids losing base functionality in overriding

```
Base::method();
```



Follow Liskov Substitution Principle (LSP)

Derived class must be usable wherever base class is expected

```
void func(Base* b);
```

LSKOV SUBSTITUTION PRINCIPLE (LSP)

LSP states: *"If S is a subtype of T, then objects of type T may be replaced by objects of type S without breaking the program."* — Barbara Liskov, 1987

- *"Objects of a derived class should be substitutable for objects of the base class without altering the correctness of the program."*
- In simple terms: **anywhere you use a base class object, you should be able to use a derived class object and everything should still work correctly.**

Simple Analogy

- Think of it like electrical plugs:
- **Base class** = standard plug socket
- **Derived class** = any device you plug in
- Every device **MUST** work when plugged into the socket. If a device breaks the socket or behaves unexpectedly → LSP violated.

```
class Bird {
public:
    virtual void fly() {
        cout << "Bird is flying\n";
    }
}

class Sparrow : public Bird {
public:
    void fly() override {
        cout << "Sparrow is flying\n";
    } // ✓ still flies, just differently
}

class Eagle : public Bird {
public:
    void fly() override {
        cout << "Eagle is soaring high\n";
    } // ✓ still flies
}

// This function works correctly with ANY Bird
void makeFly(Bird* b) {
    b->fly();
} // polymorphic call — LSP in action

int main() {
    Bird* b1 = new Sparrow();
    Bird* b2 = new Eagle();

    makeFly(b1); // ✓ Sparrow is flying
    makeFly(b2); // ✓ Eagle is soaring high
}
```

LSKOV SUBSTITUTION PRINCIPLE (LSP)

LSP VIOLATION

```
class Bird {
public:
    virtual void fly() {
        cout << "Bird is flying\n";
    }
};

class Penguin : public Bird {
public:
    void fly() override {
        throw runtime_error("Penguins can't fly!"); // ✗ LSP violated
    }
};

void makeFly(Bird* b) {
    b->fly(); // crashes when Penguin is passed!
}

int main() {
    Bird* b = new Penguin();
    makeFly(b); // ✗ program breaks — expectation violated
}
```

LSP CORRECT DESIGN

```
class Bird {
public:
    virtual void eat() {
        cout << "Bird is eating\n"; // ALL birds eat ✔
    }
};

class FlyingBird : public Bird {
public:
    virtual void fly() {
        cout << "Flying\n"; // only flying birds fly ✔
    }
};

class Sparrow : public FlyingBird {
public:
    void fly() override { cout << "Sparrow flying\n"; } // ✔
};

class Penguin : public Bird {
    // no fly() — penguins just eat, swim etc. // ✔
};
```


COMMON PITFALLS TO AVOID

⚠️ Missing virtual destructor

X `class Base { ~Base() {} }; // Non-virtual!`

✓ `class Base { virtual ~Base() {} }; // Correct!`

⚠️ Overriding without 'override' keyword

X `void draw() { ... } // Silent bug if sig changes`

✓ `void draw() override { ... } // Compiler checks`

⚠️ Slicing problem

X `Base b = derived_obj; // Slices derived part!`

✓ `Base* b = &derived_obj; // Use pointer/reference`

⚠️ Deep inheritance chains (>3 levels)

X `A→B→C→D→E→F // Complex, fragile, hard to debug`

✓ `Prefer composition or flatter hierarchy`

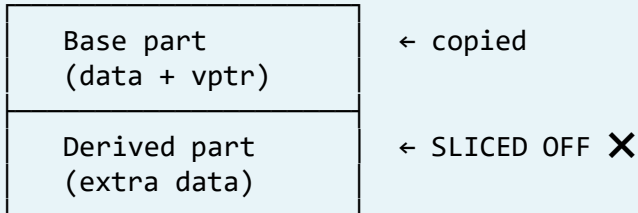
SLICING PROBLEM

When a derived class object is assigned to a base class object **by value**. The derived class's extra data and behavior gets "**sliced off**" — only the base class part is copied.

Why It Happens

When you copy by value, only the **base class portion** fits into the base class object. The derived part is simply cut away.

Derived object in memory:



```
class Animal {
public:
    string name = "Animal";
    virtual void speak() {
        cout << "Some sound\n";
    };
};

class Dog : public Animal {
public:
    string breed = "Labrador"; // extra data
    void speak() override {
        cout << "Woof! I am a " << breed << "\n";
    };
};

int main() {
    Dog dog;
    dog.name = "Rex";

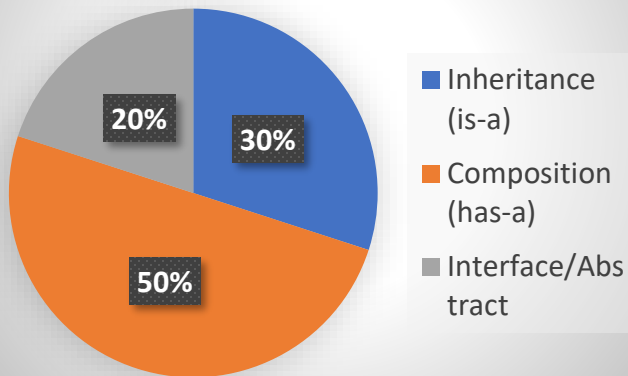
    // ✓ No slicing — pointer/reference
    Animal* ptr = &dog;
    ptr->speak();    // Woof! I am a Labrador

    // ✗ Slicing occurs — copy by value
    Animal a = dog; // dog is SLICED into a
    a.speak();      // Some sound — Dog's speak() is lost!

    cout << a.name << "\n"; // Rex    (copied)
    // cout << a.breed;      // ✗ ERROR — breed doesn't exist in Animal
}
```

COMPOSITION vs INHERITANCE: When to Use Which?

Recommended Usage in Real Projects



Aspect	Inheritance	Composition
Relationship	'is-a' (Dog is an Animal)	'has-a' (Car has an Engine)
Coupling	Tight (child depends on parent)	Loose (components independent)
Flexibility	Less flexible — hierarchy is fixed	More flexible -- swap components
Code Reuse	Automatic via inheritance chain	Via object members
Polymorphism	Natural — base pointer dispatch	Via interfaces/virtual calls
Use when	Natural 'is-a', need polymorphism	Share behavior without 'is-a'

💡 Rule of Thumb:

*Use inheritance when the **is-a** relationship is natural AND you need polymorphism. Otherwise, compose objects to share behavior without creating rigid hierarchies.*

SCOPE RESOLUTION IN INHERITANCE (:: Operator)

```
class Animal {
public:
    void speak() { cout << "Animal sound\n"; }
    void eat()    { cout << "Animal eating\n"; }
};

class Dog : public Animal {
public:
    void speak() {                // Overrides Animal::speak
        Animal::speak(); // ← Call BASE version explicitly using ::
        cout << "Woof! Woof!\n";
    }

    void demo() {
        speak();                // Calls Dog::speak (own version)
        Animal::speak();         // Calls Animal::speak (base version)
        eat();} // Calls Animal::eat (inherited, not overridden)
};

class Puppy : public Dog {
public:
    void speak() {
        Dog::speak(); // Calls Dog::speak (NOT Animal directly)
        Animal::speak(); // Can also call grandparent directly
        cout << "Yip!\n";
    }
};
```

```
int main() {
    Puppy p;
    p.speak();
}
```

```
// Output:
//   Animal sound   (from Animal::speak inside Dog::speak)
//   Woof! Woof!
//   Animal sound   (from Animal::speak directly)
//   Yip!
```

UPCASTING & DOWNCASTING

↑ Upcasting

Derived → Base ✓ Always safe

Always safe, implicit. Derived pointer/reference treated as Base.

```
Dog d("Rex");
Animal* a = &d;    // Implicit upcast ✓
Animal& ref = d;   // Reference upcast ✓
// a can now call only Animal methods
// But virtual methods still call Dog versions!
```

↓ Downcasting

Base → Derived ⚠ Potentially unsafe

Potentially unsafe — use `dynamic_cast` with RTTI for safety.

```
Animal* a = new Dog("Rex");
// Dog* d = a;           // ✗ Unsafe implicit
Dog* d = dynamic_cast<Dog*>(a); // ✓ Safe
if (d != nullptr) {
    d->fetch(); // Only if cast succeeded
}
// static_cast — no runtime check, unsafe
```

POLYMORPHISM: THE POWER OF INHERITANCE

⚙️ Compile-Time Polymorphism

Function overloading, operator overloading, templates. Resolved at compile time. Faster, no overhead.

```
int add(int); float add(float);
```

🔄 Runtime Polymorphism

Virtual functions + inheritance. Resolved at runtime via vtable. Enables truly extensible code.

```
virtual void speak();
```

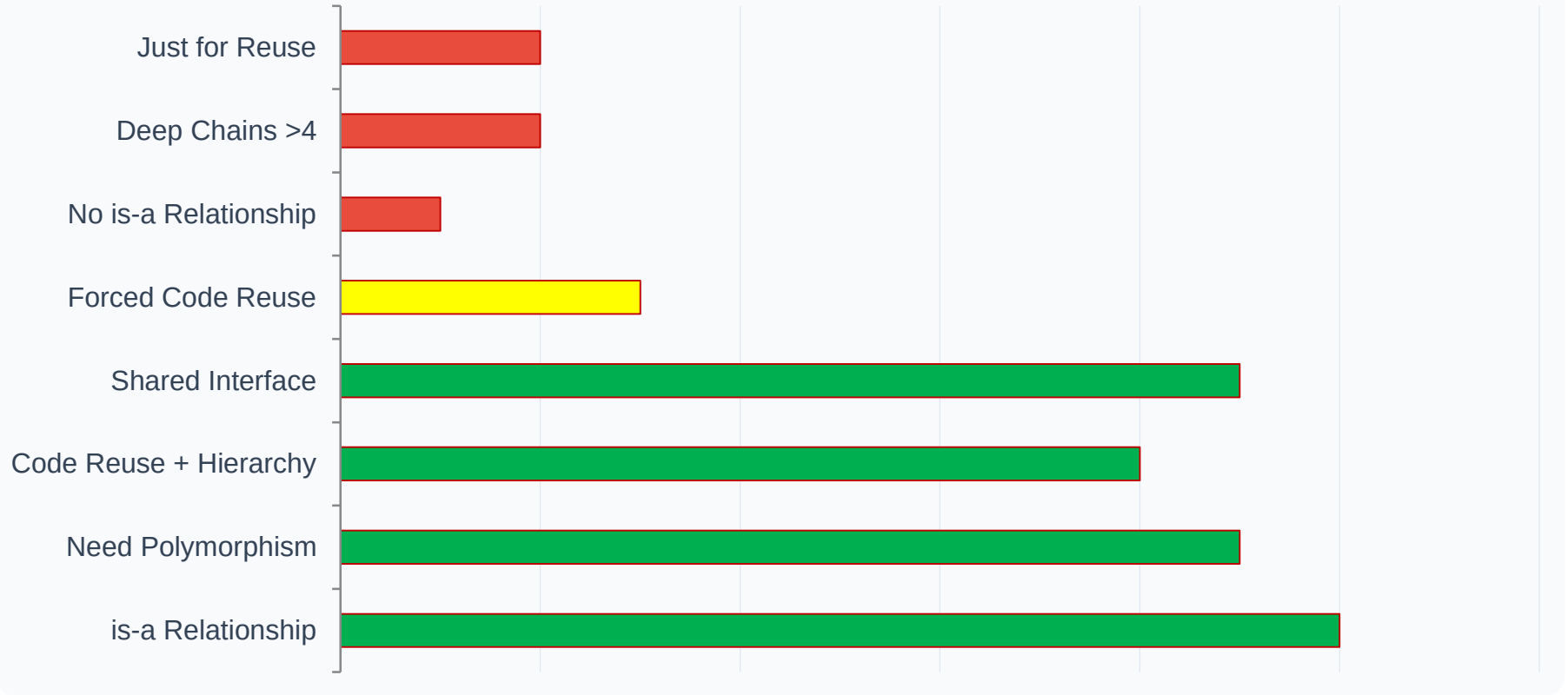
RUNTIME POLYMORPHISM IN ACTION

```
vector<Shape*> shapes;           // Array of BASE class pointers
shapes.push_back(new Circle(5));
shapes.push_back(new Rectangle(4,6));
shapes.push_back(new Triangle(3,4,5));

for (Shape* s : shapes) {
    s->draw();                    // ✔ Calls CORRECT draw() for each type – decided at RUNTIME
    s->area();                    // ✔ Calls CORRECT area() for each type
    delete s;                    // ✔ Calls CORRECT destructor (needs virtual ~Shape())
}

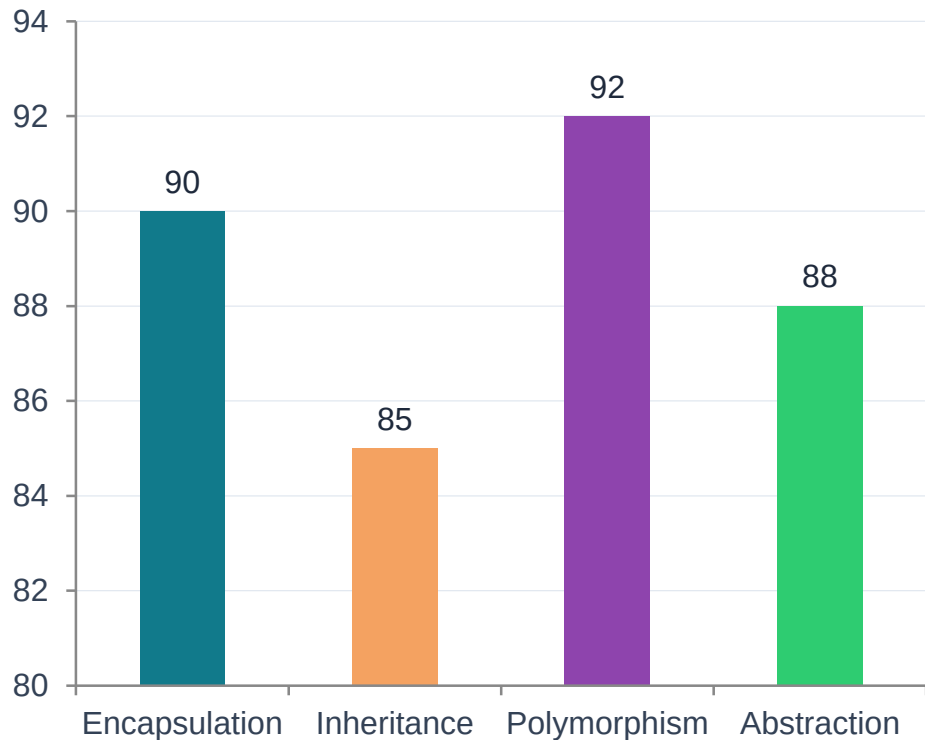
// Without virtual: ALL would call Shape::draw() – useless!
// With virtual: Each calls its own draw() – powerful!
```

WHEN TO USE (AND AVOID) INHERITANCE



INHERITANCE IN THE OOP LANDSCAPE

Four Pillars of OOP



Encapsulation

Bundles data + methods; hides internal state

→ `class with private/public`

Inheritance

Code reuse; class hierarchy; extensibility

→ `class Child : public Parent`

Polymorphism

One interface, many implementations

→ `virtual functions; override`

Abstraction

Expose only essentials; hide complexity

→ `abstract classes; interfaces`

INHERITANCE QUICK REFERENCE

Use 'is-a' to validate

class Dog : public Animal — Dog is-a Animal ✓

Use 'override' in derived classes

void draw() override — compiler checks signature match

Call base constructor in initializer list

Derived(int x) : Base(x) { } — pass params up the chain

Use virtual for runtime polymorphism

virtual void speak() — enables derived override via pointer

Base class should have virtual destructor

virtual ~Base() { } — prevents memory leaks via base pointer

protected for derived-accessible members

Base class data for derived use only goes in protected:

Destructors called in REVERSE order

Base first in construction → Derived first in destruction

Abstract class cannot be instantiated

class Shape with virtual func() = 0 → cannot do Shape s;

PRACTICE PROBLEMS

01

Animal Kingdom

Create a hierarchy: Animal → Mammal → Dog. Add a speak() virtual method overridden at each level. Demonstrate constructor chaining.

02

Shape Calculator

Design an abstract Shape class with pure virtual area() and perimeter(). Derive Circle, Rectangle, Triangle. Use polymorphism to compute total area of a mixed array.

03

Bank Account System

Create BankAccount base class. Derive SavingsAccount, CurrentAccount, LoanAccount. Override withdraw() with different rules. Use virtual balance display.

04

GUI Widget System

Build Widget (abstract) → Button, TextBox, Container → ScrollBox. Implement render() and handleClick() polymorphically. Simulate a UI event loop.

05

University System

Model Person → Student, Teacher → TeachingAssistant (hybrid). Use virtual inheritance. Implement calculatePay() for each role differently.



THANK YOU

OOP!